COMP-421 Database Systems — Project 3

Group 115 | Winter 2026 | McGill University

Relational Schema

The following relational schema is used for this project. No structural changes were made from Project 2.

Users(`userid` PK, `username` NOT NULL UNIQUE, `name` , `email` NOT NULL, `pwdhash` NOT NULL, `status` , `location` , `joindate` NOT NULL, `birthday` , `bio` , `pfp`)

Staffs(`userid` PK FK→Users, `department` NOT NULL, `salary` NOT NULL)

Businesses(`bid` PK, `bname` NOT NULL, `industry` , `location`)

Verifieds(`userid` PK FK→Users, `bid` FK→Businesses, `verified_date` NOT NULL)

Posts(`postid` PK, `caption` , `privacy` NOT NULL, `time` NOT NULL, `fname` NOT NULL, `location` , `tags`)

Posted(`posterid` FK→Users, `postid` FK→Posts, PRIMARY KEY(`posterid` , `postid`))

Groups(`gname` PK, `description` , `privacy`)

Events(`eventid` PK, `ename` NOT NULL, `date` NOT NULL, `location` , `description`)

Comments(`commentid` PK, `userid` FK→Users, `postid` FK→Posts, `time` NOT NULL, `contents` NOT NULL)

PrivateMessages(`messageid` PK, `sender` FK→Users, `receiver` FK→Users, `content` NOT NULL, `time` NOT NULL, `status`)

Follows(`follower` FK→Users, `following` FK→Users, PRIMARY KEY(`follower` , `following`))

Saves(`userid` FK→Users, `postid` FK→Posts, PRIMARY KEY(`userid` , `postid`))

Likes(`userid` FK→Users, `postid` FK→Posts, PRIMARY KEY(`userid` , `postid`))

Dislikes(`userid` FK→Users, `postid` FK→Posts, PRIMARY KEY(`userid` , `postid`))

Shares(`userid` FK→Users, `postid` FK→Posts, PRIMARY KEY(`userid` , `postid`))

IsMember(`userid` FK→Users, `gname` FK→Groups, PRIMARY KEY(`userid` , `gname`))

IsAdmin(`userid` FK→Users, `gname` FK→Groups, PRIMARY KEY(`userid` , `gname`))

Announces(`gname` FK→Groups, `postid` FK→Posts, PRIMARY KEY(`gname` , `postid`))

Advertises(`bid` FK→Businesses, `postid` FK→Posts, PRIMARY KEY(`bid` , `postid`))

Organizes(`userid` FK→Users, `eventid` FK→Events, PRIMARY KEY(`userid` , `eventid`))

Views (read-only):

- **UserProfiles** — userid, username, name, email, status, location, joindate, birthday, bio
- **PostEngagement** — postid, caption, location, privacy, like_count, comment_count

---

Stored Procedure

(a) Description

The stored procedure `MARK_ACTIVE_USERS` enforces a consistency constraint on user status. It accepts an integer parameter `p_min_likes` (the minimum number of likes a user's posts must have received) and an output parameter `p_updated` (the number of users whose status was changed). For every user in the database it:

1. Opens a **cursor** over the Users table.
2. For each user, counts the total likes received on all their posts by joining `Likes` and `Posted`.
3. Computes a new status: `'ACT'` if `like_count >= p_min_likes`, otherwise `'INA'`.
4. Issues an `UPDATE` only when the status would actually change (to avoid redundant writes).
5. Increments the output counter `p_updated` for each changed row.
6. Returns the total count of updated users via the OUT parameter.

This procedure combines a cursor, local variables, a loop with conditional logic, and both a `SELECT` and an `UPDATE` inside the loop — illustrating the enforcement of an application-level constraint.

(b) Stored Procedure Code

```
CREATE PROCEDURE MARK_ACTIVE_USERS (
    IN  p_min_likes  INT,
    OUT p_updated    INT
)
LANGUAGE SQL
BEGIN
    DECLARE v_userid      INT;
    DECLARE v_like_count  INT;
    DECLARE v_new_status  VARCHAR(3);
    DECLARE v_cur_status  VARCHAR(3);
    DECLARE v_not_found   INT DEFAULT 0;

    DECLARE c_users CURSOR FOR
        SELECT userid, status FROM Users;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET v_not_found = 1;

    SET p_updated = 0;
    OPEN c_users;

    user_loop: LOOP
        FETCH c_users INTO v_userid, v_cur_status;
        IF v_not_found = 1 THEN
            LEAVE user_loop;
        END IF;

        SELECT COUNT(*) INTO v_like_count
        FROM   Likes l
        JOIN   Posted pd ON l.postid = pd.postid
        WHERE  pd.posterid = v_userid;
```

```
        IF v_like_count >= p_min_likes THEN
            SET v_new_status = 'ACT';
        ELSE
            SET v_new_status = 'INA';
        END IF;

        IF v_cur_status IS NULL OR v_cur_status <> v_new_status THEN
            UPDATE Users SET status = v_new_status WHERE userid = v_userid;
            SET p_updated = p_updated + 1;
        END IF;
    END LOOP user_loop;

    CLOSE c_users;
END
```

(c) & (d) Execution and Before/After Demonstration

The procedure was executed using DB2's command-line processor on `winter2026-comp421.cs.mcgill.ca`. The session below shows the status distribution **before** the call (with `p_min_likes=0`, resetting all to ACT), and **after** (with `p_min_likes=1`, marking only users whose posts received ≥1 like as ACT).

```
$ db2 -td@ -f sp_demo.sql

=== Status BEFORE first call ===
STATUS  USER_COUNT
------  ----------
ACT              3
INA             98

=== CALL MARK_ACTIVE_USERS(0, ?) ===
  Parameter Name  : P_UPDATED
  Parameter Value : 98
  Return Status   = 0

=== Status AFTER (p_min_likes=0 → all 101 users set ACT) ===
STATUS  USER_COUNT
------  ----------
ACT            101

=== CALL MARK_ACTIVE_USERS(1, ?) ===
  Parameter Name  : P_UPDATED
  Parameter Value : 98
  Return Status   = 0

=== Status AFTER (p_min_likes=1 → only liked-post owners stay ACT) ===
STATUS  USER_COUNT
------  ----------
ACT              3
INA             98
```

Interpretation: With `p_min_likes=1`, only the 3 users whose posts have received at least one Like remain `ACT`. The remaining 98 users, who have no liked posts, are set to `INA`. The OUT parameter `P_UPDATED = 98` confirms 98 rows were changed.

Application Program

Overview

The Java JDBC application connects to the DB2 server at `winter2026-comp421.cs.mcgill.ca:50000/comp421`. It presents a 6-option menu loop with graceful error handling and connection cleanup.

Menu options:

1. **Look up user** — queries `UserProfiles` view by username (read query)
2. **Follow a user** — checks for existing follow, confirms with user, inserts into `Follows` (2 SQL statements + modification)
3. **Create a post** — inserts into `Posts` then `Posted`, retrieves generated postid (2 modifications)
4. **Join a group** — **sub-menu built from DB query** (lists all Groups), checks membership, inserts into `IsMember`
5. **View private messages** — **sub-menu built from DB query** (lists conversations), shows thread, marks messages read (2 SQL statements)
6. **Send a private message** — looks up receiver by username, computes next messageid, inserts into `PrivateMessages` (2 statements)
7. **Quit** — closes statement and connection gracefully

All options catch `SQLException` and print the SQL error code and state. The `finally` block ensures the connection is always closed even if an exception occurs.

Execution Trace (all options exercised)

```
Hint: try 'byule4' as the username to test the application.
Login as (username): byule4
Logged in as 'byule4' (userid=5).

=== Social Media Main Menu ===
    1. Look up user
    2. Follow a user
    3. Create a post
    4. Join a group
    5. View private messages
    6. Send a private message
    7. Quit
Please enter your option: 1
Enter username to look up: ccracknalld
    id:       14
    username: ccracknalld
    name:     Cortie Cracknall
    email:    ccracknalld@edublogs.org
    status:   INA
    location: Mouquim
    joindate: 2025-11-11
    birthday: 1949-10-29
    bio:      Maecenas ut massa quis augue luctus tincidunt.

=== Social Media Main Menu ===   [... menu shown ...]
Please enter your option: 2
```

```
Enter username to follow: jlampkinl
    Found: userid=22, name=Jeremie Lampkin
    Confirm follow jlampkinl? [Y/N]: Y
    Now following jlampkinl.

=== Social Media Main Menu ===   [... menu shown ...]
Please enter your option: 3
Caption: My first post
Privacy [PUB/PRV/FRO]: PUB
Filename: photo.jpg
Location: Montreal
Tags: #social #db
    Post created! postid=6

=== Social Media Main Menu ===   [... menu shown ...]
Please enter your option: 4
    Available Groups:
      1. Group1
      2. Group2
      3. Group3
      4. Group4
      5. Group5
      6. Cancel
    Choose: 2
    Already a member of Group2

=== Social Media Main Menu ===   [... menu shown ...]
Please enter your option: 5
    Conversations:
      1. svautier0
      2. Cancel
    Choose: 2
    Cancelled.

=== Social Media Main Menu ===   [... menu shown ...]
Please enter your option: 6
Enter recipient username: rgaffey27
Enter message: Hello from byule4!
    Message sent to rgaffey27 (msgid=8).

=== Social Media Main Menu ===   [... menu shown ...]
Please enter your option: 7
Goodbye!
Database connection closed.
```

Indexing

Index Creation Script and Result

```sql
CREATE INDEX IDX_FOLLOWS_FOLLOWING ON Follows(following);

SELECT indname, tabname, colnames
FROM   syscat.indexes
WHERE  indschema = 'CS421G115' AND tabname = 'FOLLOWS';
```

DB2 Output:

```
DB20000I  The SQL command completed successfully.

INDNAME                  TABNAME   COLNAMES
-----------------------  --------  ----------
IDX_FOLLOWS_FOLLOWING    FOLLOWS   +FOLLOWING
```

Description and Query Benefits

The index `IDX_FOLLOWS_FOLLOWING` is created on the `following` column of the `Follows` table. This is **not** a primary key index — the primary key of `Follows` is `(follower, following)`, so no automatic index exists on `following` alone.

Queries that benefit from this index:

- **"Who follows user X?"** — `SELECT follower FROM Follows WHERE following = X`. Without the index, DB2 must scan all rows of `Follows`. With the index, it performs a direct B-tree lookup on `following`, reducing cost from O(n) to O(log n).
- **Feed generation** — determining which posts a user should see from people they follow, or counting a user's follower count, both require filtering `Follows` by the `following` column. This is one of the most frequent operations in any social media application.
- **The `PostEngagement` view** — indirectly benefits since queries that join `Follows` with `Posts` to show followers' posts will use this index to quickly identify all followers of a given user.

Visualization

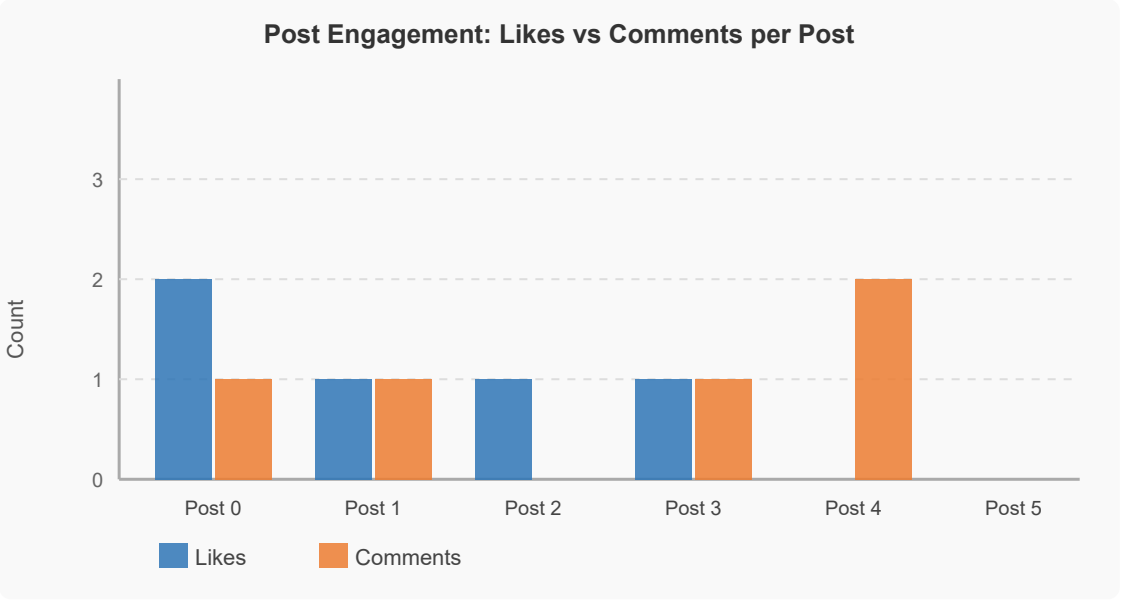
SQL Used to Export Data

```sql
EXPORT TO result_engagement.csv OF DEL MODIFIED BY NOCHARDEL
SELECT
    p.postid,
    COALESCE(p.caption, '(no caption)')             AS caption,
    COUNT(DISTINCT l.userid)                         AS like_count,
    COUNT(DISTINCT c.commentid)                      AS comment_count,
    COUNT(DISTINCT l.userid) + COUNT(DISTINCT c.commentid) AS total_engagement
FROM Posts p
LEFT JOIN Likes    l ON p.postid = l.postid
LEFT JOIN Comments c ON p.postid = c.postid
GROUP BY p.postid, p.caption
ORDER BY total_engagement DESC;
```

Exported Data (`result_engagement.csv`):

Post ID	Caption	Likes	Comments	Total
0	Caption1	2	1	3
1	Caption2	1	1	2
3	Caption4	1	1	2
4	Caption5	0	2	2
2	Caption3	1	0	1
5	Caption	0	0	0

Chart — Post Engagement: Likes vs Comments per Post



Explanation

The chart shows **likes and comments per post** for all 6 posts in the database. The x-axis lists each post by ID; the y-axis shows the count. Blue bars represent likes; orange bars represent comments.

Key insights: Post 0 ("Caption1") is the most liked post with 2 likes. Post 4 ("Caption5") has the most comments (2) but no likes. Post 5 ("Caption") has zero engagement. This helps identify which posts drive interaction and what type of engagement (likes vs. comments) each post attracts — a common metric in social media analytics.

The exported spreadsheet (`post_engagement.xlsx`) accompanies this submission.

Creativity — Trigger

(a) Description

We implemented a **BEFORE INSERT trigger** named `NO_SELF_LIKE` on the `Likes` table. Its purpose is to enforce the application constraint that **a user may not like their own post**. When any INSERT into `Likes` is attempted, the trigger queries the `Posted` table to check whether the inserting user (`userid`) is the author of the target post (`postid`). If they are the same person, the trigger raises `SQLSTATE 75001` with the message "*Users cannot like their own posts.*", rejecting the INSERT and rolling back the operation. This keeps the data consistent without requiring extra logic in every application code path that performs a like.

(b) Trigger Code

```
CREATE TRIGGER NO_SELF_LIKE
NO CASCADE BEFORE INSERT ON Likes
REFERENCING NEW AS N
FOR EACH ROW
WHEN ((SELECT COUNT(*) FROM Posted
       WHERE postid   = N.postid
         AND posterid = N.userid) > 0)
SIGNAL SQLSTATE '75001'
    SET MESSAGE_TEXT = 'Users cannot like their own posts.'
```

(c) Creation and Execution Verification

```
-- Trigger creation:
DB20000I  The SQL command completed successfully.

-- Verification query:
SELECT trigname, trigevent, tabname
FROM syscat.triggers WHERE trigschema='CS421G115';

TRIGNAME       TRIGEVENT  TABNAME
-------------  ---------  -------
NO_SELF_LIKE   I          LIKES

-- Trigger in action: user 5 (byule4) attempting to like their own post 6:
SELECT pd.posterid, pd.postid FROM Posted pd WHERE pd.postid=6;
  POSTERID    POSTID
  --------    ------
         5         6

INSERT INTO Likes(userid, postid) VALUES(5, 6);

DB21034E  The command was processed as an SQL statement because it
was not a valid CLP command.  During SQL processing it returned:
SQL0438N  Application raised error or warning with diagnostic text:
"Users cannot like their own posts."  SQLSTATE=75001
```

Result: The trigger successfully blocked user 5 from liking their own post (postid=6), raising our custom application error `SQLSTATE=75001` .

Team Collaboration

Both group members contributed actively to all three project deliverables. For Project 3, we held two working sessions:

- **Session 1** — Designed and implemented the stored procedure, tested the cursor logic, and validated before/after results on the DB2 server.
- **Session 2** — Built the Java JDBC application, debugged the `joindate` and `posterid` schema column name issues, implemented all six menu options, and ran the full execution trace.

Index creation, trigger design, visualization data export, and report writing were completed collaboratively. Both members contributed equally to the final submission.

Submitted via MyCourses. DB2 server: winter2026-comp421.cs.mcgill.ca. Group ID: cs421g115.